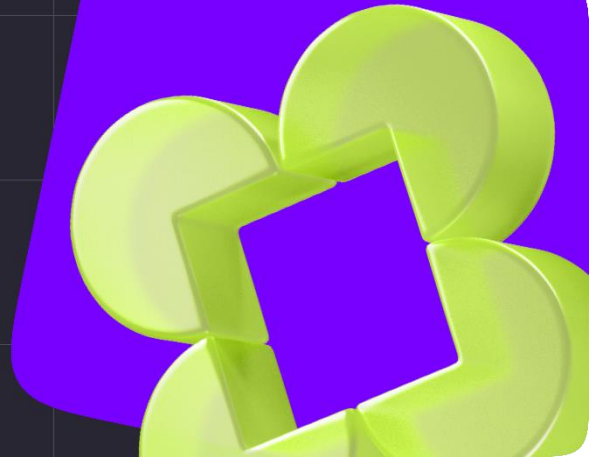


Постридер

SQL для начинающих
Занятие 5



Оконные функции

Мы поговорили с вами об оконных функциях.

Оконные функции — это специальные функции в SQL, которые:

- Выполняют вычисления по группе строк, связанных с текущей строкой;
- Не «сворачивают» строки (в отличие от GROUP BY);
- Сохраняют исходное количество строк, добавляя к каждой новое вычисленное значение.

Синтаксис

Вот так выглядит общий синтаксис:

```
<оконная_функция>() OVER (  
    [PARTITION BY столбец1, ...] -- разбивает данные на группы («окна»)  
    [ORDER BY столбец2, ...] -- задаёт порядок внутри окна  
    [ROWS/RANGE BETWEEN ...] -- определяет рамки окна (необязательно)  
)
```

Т.е. оконные функции позволяют выполнять вычисления по «окну» строк, связанному с текущей строкой, не сворачивая исходный набор данных. Каждая строка остаётся в результате, но к ней добавляется вычисленное значение.

Основные типы оконных функций

Функция	Назначение	Пример использования
SUM(), AVG(), COUNT() и др.	Агрегатные функции в окне	Накопительный итог, скользящее среднее
LAG(столбец)	Значение из предыдущей строки	Сравнить платёж с предыдущим
LEAD(столбец)	Значение из следующей строки	Прогнозировать следующий платёж
ROW_NUMBER()	Уникальная нумерация строк	Пронумеровать платежи по дате
RANK() / DENSE_RANK()	Ранжирование с/без пропусков	Топ клиентов по сумме платежей
FIRST_VALUE() / LAST_VALUE()	Первое/последнее значение в окне	Самый первый платёж клиента

NTILE(n)	Деление на n равных групп	Разделить клиентов на квартили
----------	---------------------------	--------------------------------

Пример

```
SELECT booking_id, renter_id, check_in_date,
RANK() OVER (PARTITION BY renter_id ORDER BY check_in_date) AS booking_rank FROM
bookings;
```

Этот запрос делает следующее:

- Группирует строки по **renter_id** — то есть отдельно обрабатывает бронирования каждого арендатора (**PARTITION BY renter_id**).
- Внутри каждой группы сортирует записи по дате заезда (**ORDER BY check_in_date**).
- Присваивает ранг (**RANK()**) каждой записи в пределах своей группы:
 - Первая (самая ранняя) дата заезда получает ранг 1,
 - Если у одного арендатора несколько бронирований с одинаковой датой заезда, они получают одинаковый ранг,
 - Следующая более поздняя дата получит ранг с пропуском (например: 1, 1, 3 — если первые два совпадают).

Когда использовать оконные функции?

- Нужно видеть исходные строки + агрегированные значения одновременно.
- Требуется сравнение с соседними записями (например, рост/падение).
- Надо посчитать накопленную сумму, ранг, скользящее среднее и т.п.

Важно помнить

- **PARTITION BY** — аналог **GROUP BY**, но он не уменьшает число строк.
- **ORDER BY** внутри **OVER()** нужен для корректной работы **LAG, LEAD**, накопительных сумм.
- Без **PARTITION BY** окно захватывает все строки результата (то есть считает все строки как одно окно).